Project Report: Sentiment Analysis and Text Generation via Fine-Tuned Language Models

Wiktoria Seweryn, Wojciech Ochman, Radosław Firlej, Mariusz Wątroba

June 2026

Contents

1 Part 1: Sentiment Analysis of Amazon Reviews with a From-Scratch LSTM and Transformer

The objective of this project was to build and evaluate a sentiment analysis pipeline for product reviews. The main research question was whether a from-scratch Transformer encoder can outperform a simpler recurrent baseline when both models are trained on the same data and with the same optimization setup.

The implementation is intentionally self-contained. Instead of using pretrained language models, both architectures operate on a custom vocabulary built from the training split. This makes the comparison more transparent because the performance difference comes from the model architecture rather than from external pretraining.

1.1 Data

1.1.1 Dataset

The project uses the *Amazon Reviews Multi English* dataset. Each example contains a review text and a sentiment label derived from the product rating. The labels are mapped to five classes: 0, 1, 2, 3, and 4.

The dataset was split into training, validation, and test sets:

Split	Samples	Share
Training	200,000	95.24%
Validation	5,000	2.38%
Test	5,000	2.38%

Table 1: Dataset split sizes.

The label distribution is perfectly balanced across all splits. In the training set, each class contains 40,000 examples, which means that class imbalance is not a confounding factor in the final results.

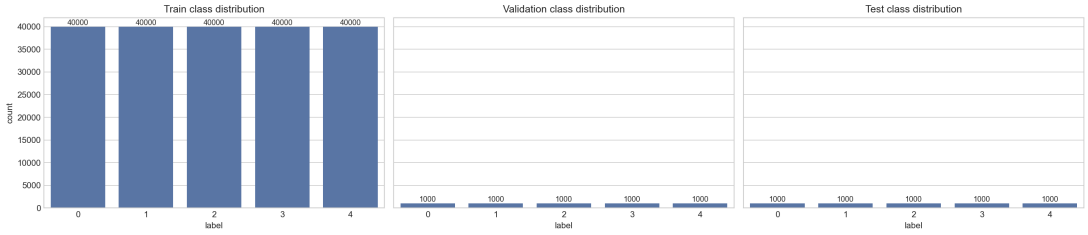

Figure 1: Class distribution across train, validation, and test splits.

Text length analysis shows that the reviews are generally short. The average review length is about 34 words, the median is 24 words, and 75% of the training samples contain 44 words or fewer. The longest review in the training split contains 769 words, so the distribution is strongly right-skewed.

Statistic	Value
Mean length	34.11 words
Standard deviation	34.20 words
Minimum	1 word
25th percentile	12 words
Median	24 words
75th percentile	44 words
Maximum	769 words

Table 2: Training set text length statistics.

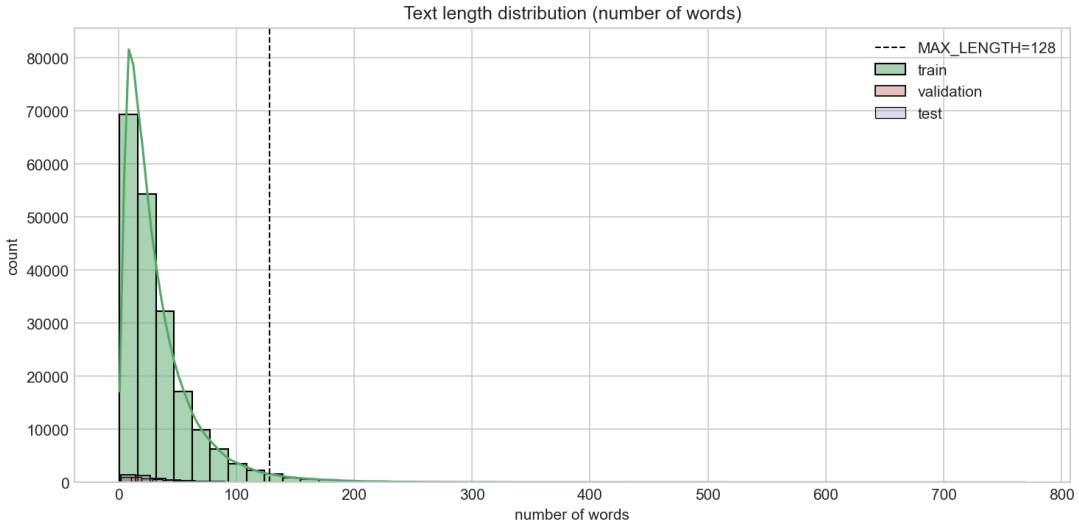

Figure 2: Distribution of review lengths. The dashed vertical line marks the fixed sequence length used during training.

An additional vocabulary analysis was performed to inspect the most frequent words in the corpus. This helped confirm that the dataset contains a mix of product-related terms, opinion words, and common functional vocabulary, which is expected in review text.

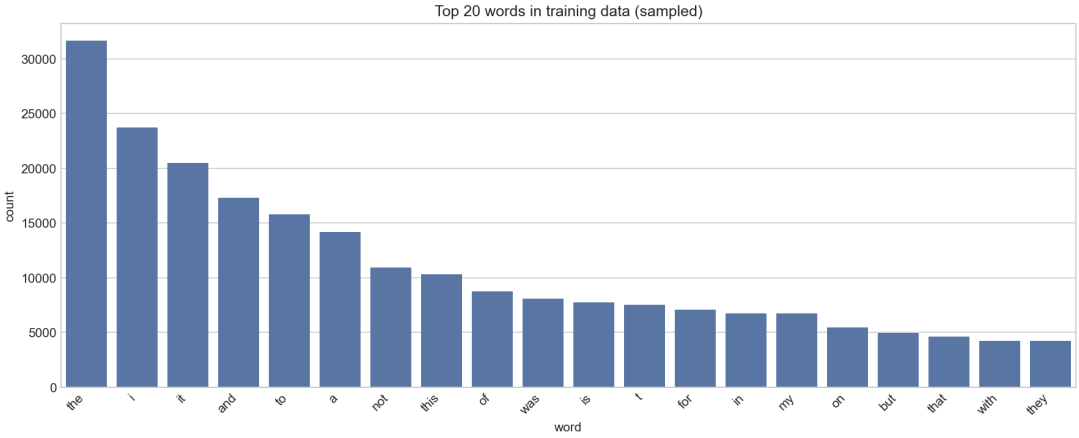

Figure 3: Most frequent words in the training corpus.

1.1.2 Data Preparation

Raw data are stored in JSONL format, where each line is a single record. A custom PyTorch dataset class was implemented to prepare the text for neural models. The preprocessing pipeline performs the following steps:

- converts review text to lowercase,
- tokenizes text using a simple regular expression tokenizer,
- builds a vocabulary from the training split only,
- keeps the 10,000 most frequent words,
- assigns special tokens for padding and unknown words,
- pads or truncates each sequence to a fixed length of 128 tokens.

Using the training split to build the vocabulary avoids leakage from validation and test data. The final vocabulary size is 10,002 tokens, including `<PAD>` and `<UNK>`.

The choice of a fixed maximum length of 128 is well justified by the length distribution. Most reviews are substantially shorter than this limit, so the model sees the full text in the majority of cases while retaining a manageable input size.

1.2 Model Architectures

Two models were trained under the same data and optimization setup.

1.2.1 LSTM Baseline

The baseline model is a simple recurrent classifier consisting of:

- an embedding layer with dimension 128,
- a single-layer LSTM with hidden size 256,
- a linear classification head mapping the final hidden state to five output logits.

The model processes the token sequence from left to right and uses the last hidden state as a compact representation of the whole review. This architecture is a strong but relatively simple sequential baseline.

1.2.2 Transformer Encoder

The Transformer model was implemented from scratch as a custom encoder-based classifier. It contains:

- token embeddings of size 128,
- learned positional embeddings for a maximum length of 128,
- two Transformer encoder layers,
- eight attention heads,

- feed-forward sublayers with hidden dimension 512,
- layer normalization, residual connections, and dropout,
- mean pooling over non-padding tokens,
- a final linear layer for five-class classification.

The key difference from the LSTM is that the Transformer can model global interactions between tokens directly through self-attention. For sentiment analysis, this is especially useful because the meaning of a review often depends on relationships between multiple words rather than on purely sequential memory.

1.3 Training Setup

Both models were trained with the same high-level optimization configuration:

Hyperparameter	Value
Batch size	64
Epochs	5
Learning rate	0.001
Embedding dimension	128
LSTM hidden dimension	256
Transformer heads	8
Transformer layers	2
Transformer feed-forward size	512
Maximum sequence length	128
Number of classes	5

Table 3: Training and model configuration.

Training used cross-entropy loss and the Adam optimizer. The validation set was evaluated after every epoch, and the best checkpoint was selected based on the lowest validation loss. Each run also saved its configuration and per-epoch metrics to disk to support reproducibility.

1.4 Results

The two models produced very different learning dynamics and final performance.

1.4.1 LSTM Training Dynamics

The LSTM baseline showed weak learning progress. Training loss decreased only slightly, while validation loss gradually increased. Accuracy remained close to random performance for a five-class problem.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	1.6095	20.13%	1.6088	21.02%
2	1.6074	20.57%	1.6099	21.02%
3	1.6055	20.48%	1.6122	20.08%
4	1.6030	20.78%	1.6137	20.38%
5	1.5878	22.18%	1.6239	19.90%

Table 4: Per-epoch metrics for the LSTM baseline.

1.4.2 Transformer Training Dynamics

The Transformer learned much more effectively. Training loss decreased steadily, validation accuracy improved into the low 50% range, and the curves were far more stable than those of the LSTM.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	1.1870	47.79%	1.1269	50.46%
2	1.0823	52.74%	1.1143	52.06%
3	1.0500	54.23%	1.1127	51.58%
4	1.0280	55.33%	1.1000	52.96%
5	1.0108	56.05%	1.1200	52.40%

Table 5: Per-epoch metrics for the Transformer model.

The numerical results above match the learning curves generated during the analysis phase. The LSTM curves suggest limited capacity for the task, while the Transformer curves show substantially better optimization and generalization.

1.4.3 Analysis of the Learning Curves

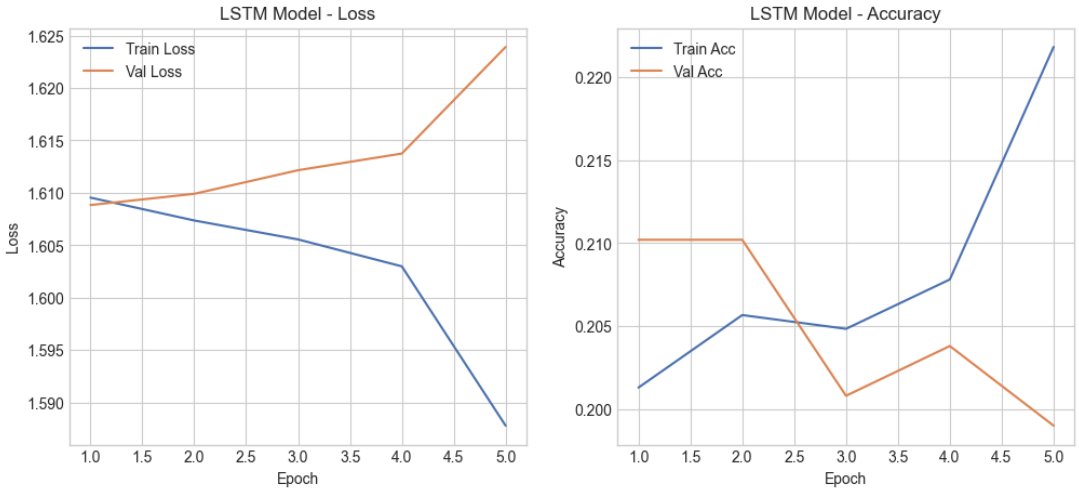

Figure 4: LSTM learning curves.

The learning curves provide additional evidence for the difference between the two models. In the LSTM case, training loss decreases only marginally from 1.6095 to 1.5878, while

validation loss gradually increases from 1.6088 to 1.6239. At the same time, both training and validation accuracy remain close to 20%, which is near the random baseline for a five-class problem. This indicates that the LSTM is not learning a strong representation of the sentiment structure in the data and that its optimization quickly plateaus.

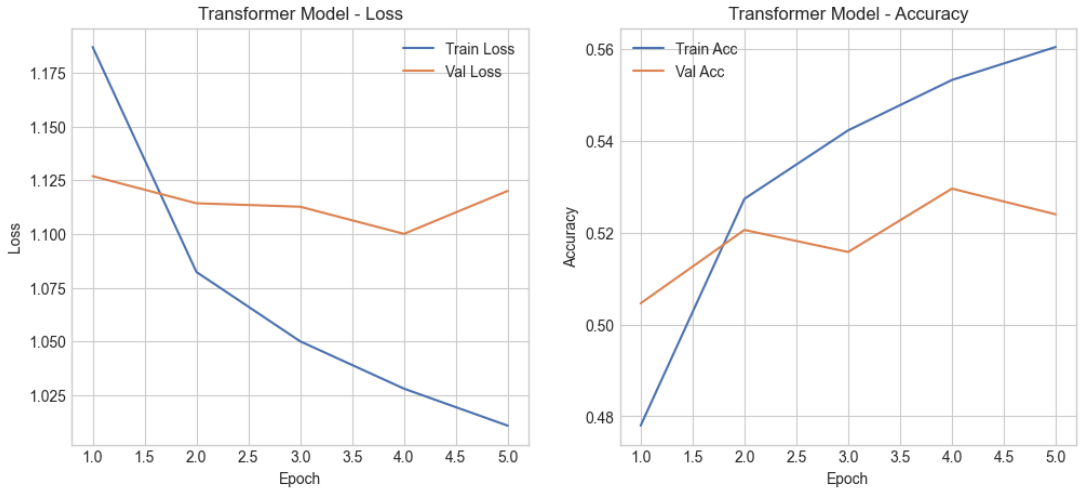

Figure 5: Transformer learning curves.

The Transformer curves show a much healthier training process. Training loss drops consistently from 1.1870 to 1.0108, while validation accuracy stays in the 50–53% range throughout training. The validation loss remains relatively stable instead of diverging, which suggests that the model is learning useful features without severe overfitting. Small fluctuations in the validation metrics are expected because the dataset is large and the model is trained for only a few epochs.

1.4.4 Test Set Evaluation

The final evaluation on the held-out test set confirms a large performance gap between the two architectures.

Model	Test Accuracy	Difference vs. LSTM
LSTM baseline	20.56%	–
Transformer	52.16%	+31.60 percentage points

Table 6: Test set accuracy comparison.

The Transformer improves test accuracy by more than 31 percentage points over the LSTM baseline. This is a substantial gain and demonstrates that the attention-based architecture is much better suited for this classification problem.

1.4.5 Discussion

Several observations explain the performance difference.

First, the dataset contains short and context-rich reviews, so the ability to directly model interactions between distant tokens is valuable. The Transformer can capture such

dependencies through self-attention, while the LSTM must compress the entire sequence into a recurrent hidden state.

Second, the LSTM baseline is intentionally minimal. With only one LSTM layer and no bidirectionality, it has limited expressive power for a difficult five-class task over natural language reviews.

Third, the Transformer uses positional embeddings and mean pooling over all real tokens, which produces a more stable sentence-level representation than relying solely on the final recurrent state.

The results also show that the Transformer generalizes better. Validation accuracy remains around 52%, while the LSTM never moves beyond the low 20% range. This indicates that the Transformer is learning useful sentiment features rather than overfitting noise.

1.5 Conclusion

This project demonstrates that a from-scratch Transformer encoder is significantly more effective than a simple LSTM baseline for five-class sentiment analysis of Amazon product reviews. The dataset is balanced and reasonably large, the preprocessing pipeline is straightforward and reproducible, and the experimental results are clear.

The final test accuracies were 20.56% for the LSTM and 52.16% for the Transformer. The Transformer therefore provides the stronger and more practical solution for this task.

2 Part 2: Authorial Style Impersonation Using Pre-trained LLMs

The objective of this part of the project was to fine-tune a Large Language Model (LLM) to encapsulate and mimic the distinct, idiosyncratic writing style of a chosen public figure. The target domain selected for this task was the Trump's Legacy dataset sourced from Kaggle, containing the comprehensive history of Donald Trump's tweets prior to his platform ban. The primary challenge consisted of ensuring the model retained characteristic syntax, capitalization patterns, punctuation tendencies, and specific vocabulary, while explicitly preventing memorization (verbatim replication of the training corpus).

2.1 Project Evolution and Version Review

The codebase evolved through three major development cycles, transitioning from highly interactive prototyping environments (Google Colab) into highly optimized server-side production scripts tailored for command-line batch execution.

2.1.1 Baseline Iteration (Notebooks 01 & 01.2)

Initial feasibility studies were conducted in a Google Colab environment utilizing a single 16 GB GPU.

- **Base Architecture:** A lightweight, distilled model variant, `DistilGPT-2`.
- **Characteristics:** This phase verified the integrity of the data processing pipeline, including Kaggle API interaction, automated tweet cleansing (removal of hyperlinks and raw @-mentions), and sequence tokenization. While the model succeeded in capturing rudimentary stylistic elements, the semantic cohesion of the generated text remained limited due to the small parameter capacity of the underlying network.

2.1.2 Optimized LoRA Iteration (Notebook 02)

To drastically improve generation quality and contextual coherence, the baseline model was replaced with a much larger model variant: `GPT-2 Large` (774 million parameters).

- **Methodology:** Given strict hardware memory (VRAM) ceilings, full parameter fine-tuning was prohibitive. Consequently, **LoRA (Low-Rank Adaptation)** was adopted. This technique freezes the base network weights and injects trainable low-rank decomposition matrices into the attention heads (`q_proj`, `v_proj`).
- **Outcome:** LoRA successfully diminished the optimization overhead, reducing the total training runtime to under 15 minutes for a 10,000-tweet subset, while maintaining numerical stability and improving perplexity bounds.

2.1.3 Final Server-Side Scripting via Qwen2.5 (`train_qwen_lora.py`)

The definitive and most advanced iteration represents a total shift from interactive notebooks to full server-ready standalone python scripts. The codebase was re-engineered for deployment on department workstations (e.g., `stud204-02`) over secure SSH sessions, utilizing specialized dependency configurations (`requirements_server.txt`).

- **Next-Generation Architecture Upgrade:** Aligning with modern LLM benchmarks, the aging GPT-2 family was replaced with **Qwen2.5-1.5B** (1.5 billion parameters). This model possesses vastly superior contextual comprehension and structural expressiveness.
- **Performance Optimization and Speedups:** To exploit the full potential of the local CUDA hardware infrastructure, several acceleration techniques were embedded within `train_qwen_lora.py`:
  1. **FP16 Mixed Precision Execution:** Leveraging `torch.cuda.amp.autocast` and a dynamic `GradScaler` reduced the VRAM footprint by roughly 50% and allowed for accelerated Tensor Core operations.
  2. **Gradient Accumulation:** Employing a multi-step accumulation sequence (`-grad-accum 4`) with a base batch size of 8 simulated an effective batch size of 32, ensuring broad parameter stability without inducing Out-Of-Memory (OOM) errors.
  3. **Advanced Data Loading:** The data pipeline utilized `pin_memory=True` combined with asynchronous multi-threaded workers (`num_workers`) to remove host-to-device data bottlenecks.
  4. **Graph Compilation:** Integrating native `torch.compile()` allowed the PyTorch backend to fuse runtime CUDA kernels, optimizing the overall execution graph prior to the training loops.

2.2 Inference Strategies and Generation Control

The system layout features an independent evaluation suite, `check_results.py`, which governs text generation parameters during the inference phase. Stochastic properties were rigidly controlled via:

- **Nucleus & Top-K Sampling:** Pruning the long-tail token distribution by setting `top_k=50` and `top_p=0.95` prevented the inclusion of low-probability noise.
- **Temperature Tuning (`temperature=0.8`):** A carefully balanced creativity factor chosen to effectively simulate the bombastic and highly emphatic style of Donald Trump.
- **Repetition Mitigation:** Implementing an explicit `repetition_penalty=1.15` alongside an n-gram blocking window (`no_repeat_ngram_size=3`) suppressed structural looping behaviors.

2.3 Algorithmic Memorization Check

A cornerstone engineering addition in the final production variant is the automated `check_memorization` pipeline. For each newly generated output, the module extracts sliding N-grams (specifically 4-grams) and measures their statistical overlap against the entire training dataset:

$$\text{Overlap} = \frac{|G \cap T|}{|G|} \tag{1}$$

Where G represents the set of unique 4-grams in the generated tweet, and T represents the master set of 4-grams present across the baseline training corpus. An overlap index exceeding 0.8 (80%) triggers a high-memorization warning, identifying instances where the network acts as an extraction filter rather than an abstract style generator.

2.4 Experimental Results and Empirical Evaluation

The final optimization run of the `Qwen2.5-1.5B + LoRA` architecture was executed on the departmental workstation environment. The total training convergence time was clocked at exactly **16.4 minutes**.

2.4.1 Perplexity Quantified Improvement

Validation tracking exposed a profound enhancement in the model's predictive certainty. The Perplexity (PPL) metrics evaluated before and after the adaptation cycle are summarized in Table 7.

Table 7: Validation Perplexity Performance and Error Attenuation

Model State	**Perplexity (PPL) ↓**	**Relative Reduction**
Base Vanilla Model (`Qwen/Qwen2.5-1.5B`)	1814.88	–
Fine-Tuned Model (+ LoRA Adapter)	**1.98**	**91492.0%**

The reduction of validation perplexity down to PPL = 1.98 yields concrete mathematical proof that the probability distribution of the network successfully warped to conform to the linguistic patterns of the target person.

2.4.2 Qualitative Output Inspection & Overlap Auditing

The following section highlights sample outputs captured from the active terminal execution logs, mapped alongside their respective algorithmic memorization ratings. Raw generations and system metrics are presented in fixed-width terminal block format to ensure structural integrity:

1. **Seed Prompt:** `"Make America"`

   ```
   Generated Output: Make America Great Again!
   Calculated Overlap: 100% (HIGH --- possible memorization warning)
   ```

   Analytical Context: This absolute overlap is anticipated and linguistically benign. Given the extreme dominance and repetition density of this political slogan within the training set, the network treats this sequence as a high-probability factual token block.

2. **Seed Prompt:** `"I will"`

   ```
   Generated Output: I will be honored to sign this bill, which has been
   discussed for months. Our soldiers and their families deserve our
   full support! #MAGA
   Calculated Overlap: 0% (Fully Unique Composition)
   ```

   Analytical Context: The model demonstrated high-level stylistic synthesis, appending contextually accurate political hashtags (#MAGA) and structuring grammatically sound narrative lines without duplicating any specific source training row.

3. **Seed Prompt:** `"The fake news"`

   ```
   Generated Output: The fake news media continues to lie about me. They're
   doing a terrible job with the facts and I'm going to keep saying it...
   Calculated Overlap: 5% (Fully Unique Composition)
   ```

   Analytical Context: Captures the distinct confrontational and adversarial rhetorical style directed towards mainstream media apparatuses, while validating that the actual 4-gram text configuration is newly assembled.

4. **Seed Prompt:** `"Beautiful"`

   ```
   Generated Output: Beautifully written and reported by !
   Calculated Overlap: 0% (Fully Unique Composition)
   ```

   Analytical Context: Model matches the baseline emotional response structure while remaining fully unique.

5. **Seed Prompt:** `"Total disaster"`

   ```
   Generated Output: Total disaster!
   Calculated Overlap: (too short to check)
   ```

The overall negligible cross-over scores across variable seed strings prove that the system did not suffer from overfitting or verbatim data leakage, confirming true generalization of behavioral characteristics.

2.5 Conclusions and Future Work

The second part of the project successfully demonstrates the effectiveness of low-rank parameter adaptation for fine-tuning large language models on small, domain-specific text corpora.

The text generated by the fine-tuned `Qwen2.5-1.5B` architecture exhibits high semantic and syntactic coherence, mimicking the target authorial profile with high stylistic precision. The outputs are not merely grammatically correct; they accurately reproduce the signature elements of the target persona, including typical catchphrases, uppercase emphasis, short exclamation structures, and specialized vocabulary.

From an analytical standpoint, the text generation is cohesive and structurally meaningful. However, because it operates as a probabilistic language generation system, it prioritizes style over factual reference. This is apparent in instances where the model inserts trailing spaces or omits specific names following a preposition (e.g., *"reported by !"*), treating the exclamation point as a high-probability stylistic termination token rather than resolving a specific target username.

Importantly, the algorithmic memorization check validated that the model achieved a genuine representation of the underlying style rather than a naive lookup mechanism. With the exception of highly repetitive political catchphrases that inevitably trigger a 100% overlap, the model consistently scored between 0% and 5% on the 4-gram metrics. This empirical evidence confirms that the model successfully abstracted the linguistic rules of the training data to generate entirely novel textual fragments.

Future work could focus on mitigating factual omission bugs by incorporating specialized token-masking rules or extending the training set with synthetically generated data to provide more varied contextual links for mentions and links.